Reguläre Sprachen, Ausdrucksstärke

BC George (HSBI)

Weitere Unterlagen

- Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke

1 Motivation

1.1 Was muss ein Compiler wohl als erstes tun?

Hier entsteht ein Tafelbild.

1.2 Themen für heute

- Endliche Automaten
- Reguläre Sprachen

2 Endliche Automaten

2.1 Deterministische endliche Automaten

Def.: Ein **deterministischer endlicher Automat** (DFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \to Q$, δ kann partiell sein
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

2.2 Nichtdeterministische endliche Automaten

Def.: Ein **nichtdeterministischer endlicher Automat** (NFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) Übergangsfunktion $(Q \times \Sigma) \to Q$
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

2.3 Akzeptierte Sprachen

Def.: Sei A ein DFA oder ein NFA. Dann ist **L(A)** die von A akzeptierte Sprache, d. h.

$L(A) = \{$Wörter $w \mid \delta^*(q_0, w) \in F\}$

2.4 Wozu NFAs im Compilerbau?

Pattern Matching (Erkennung von Schlüsselwörtern, Bezeichnern, ...) geht mit NFAs.

NFAs sind so nicht zu programmieren, aber:

Satz: Eine Sprache L wird von einem NFA akzeptiert $\Leftrightarrow$ L wird von einem DFA akzeptiert.

D. h. es existieren Algorithmen zur

- Umwandlung von NFAs in DFAS
- Minimierung von DFAs

3 Reguläre Sprachen

3.1 Reguläre Ausdrücke

Def.: Induktive Definition von **regulären Ausdrücken** (regex) und der von ihnen repräsentierten Sprache **L**:

- Basis:
  - ϵ und $\emptyset$ sind reguläre Ausdrücke mit $L(\epsilon) = \{\epsilon\}$, $L(\emptyset) = \emptyset$
  - Sei a ein Symbol $\Rightarrow a$ ist ein regex mit $L(a) = \{a\}$
- Induktion: Seien $E,\ F$ reguläre Ausdrücke. Dann gilt:
  - $E + F$ ist ein regex und bezeichnet die Vereinigung $L(E + F) = L(E) \cup L(F)$
  - EF ist ein regex und bezeichnet die Konkatenation $L(EF) = L(E)L(F)$
  - E^* ist ein regex und bezeichnet die Kleene-Hülle $L(E^*) = (L(E))^*$
  - (E) ist ein regex mit $L((E)) = L(E)$

Vorrangregeln der Operatoren für reguläre Ausdrücke: *, Konkatenation, +

3.2 Formale Grammatiken

Def.: Eine *formale Grammatik* ist ein 4-Tupel $G = (N, T, P, S)$ aus

- N: endliche Menge von **Nichtterminalen**
- T: endliche Menge von **Terminalen**, $N \cap T = \emptyset$
- $S \in N$: **Startsymbol**

- *P*: endliche Menge von **Produktionen** der Form

$X \to Y$ mit $X \in (N \cup T)^* N (N \cup T)^*, Y \in (N \cup T)^*$

3.3 Ableitungen

Def.: Sei $G = (N, T, P, S)$ eine Grammatik, sei $\alpha A \beta$ eine Zeichenkette über $(N \cup T)^*$ und sei $A \to \gamma$ eine Produktion von G.

Wir schreiben: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ($\alpha A \beta$ leitet $\alpha \gamma \beta$ ab).

Def.: Wir definieren die Relation $\overset{*}{\Rightarrow}$ induktiv wie folgt:

- Basis: $\forall \alpha \in (N \cup T)^* \alpha \overset{*}{\Rightarrow} \alpha$ (Jede Zeichenkette leitet sich selbst ab.)
- Induktion: Wenn $\alpha \overset{*}{\Rightarrow} \beta$ und $\beta \Rightarrow \gamma$ dann $\alpha \overset{*}{\Rightarrow} \gamma$

Def.: Sei $G = (N, T, P, S)$ eine formale Grammatik. Dann ist $L(G) = \{$Wörter w über $T \mid S \overset{*}{\Rightarrow} w\}$ die von G erzeugte Sprache.

3.4 Reguläre Grammatiken

Def.: Eine **reguläre (oder type-3-) Grammatik** ist eine formale Grammatik mit den folgenden Einschränkungen:

- Alle Produktionen sind entweder von der Form
  - $X \to aY$ mit $X \in N, a \in T, Y \in N$ (*rechtsreguläre* Grammatik) oder
  - $X \to Ya$ mit $X \in N, a \in T, Y \in N$ (*linksreguläre* Grammatik)
- $X \to \epsilon$ ist erlaubt

3.5 Reguläre Sprachen und ihre Grenzen

Satz: Die von endlichen Automaten akzeptiert Sprachklasse, die von regulären Ausdrücken beschriebene Sprachklasse und die von regulären Grammatiken erzeugte Sprachklasse sind identisch und heißen **reguläre Sprachen**.

Reguläre Sprachen

- einfache Struktur
- Matchen von Symbolen (z. B. Klammern) nicht möglich, da die fixe Anzahl von Zuständen eines DFAs die Erkennung solcher Sprachen verhindert.

3.6 Wozu reguläre Sprachen im Compilerbau?

- Reguläre Ausdrücke
  - definieren Schlüsselwörter und alle weiteren Symbole einer Programmiersprache, z. B. den Aufbau von Gleitkommazahlen
  - werden (oft von einem Generator) in DFAs umgewandelt
  - sind die Basis des *Scanners* oder *Lexers*

3.7 Ein Lexer ist mehr als ein DFA

- Ein **Lexer**
  - wandelt mittels DFAs aus regulären Ausdrücken die Folge von Zeichen der Quelldatei in eine Folge von sog. Token um
  - bekommt als Input eine Liste von Paaren aus regulären Ausdrücken und Tokennamen, z. B. ("while", WHILE)
  - Kommentare und Strings müssen richtig erkannt werden. (Schachtelungen)
  - liefert Paare von Token und deren Werte, sofern benötigt, z. B. (WHILE, _), oder (IDENTIFIER, "radius") oder (INTEGERZAHL, "334")

3.8 Wie geht es weiter?

- Ein **Parser**
  - führt mit Hilfe des Tokenstreams vom Lexer die Syntaxanalyse durch
  - basiert auf einer sog. kontextfreien Grammatik, deren Terminale die Token sind
  - liefert die syntaktische Struktur in Form eines Ableitungsbaums (**syntax tree**, **parse tree**), bzw. einen **AST** (abstract syntax tree) ohne redundante Informationen im Ableitungsbaum (z. B. Semikolons)
  - liefert evtl. Fehlermeldungen

4 Wrap-Up

4.1 Wrap-Up

- Definition und Aufgaben von Lexern
- DFAs und NFAs
- Reguläre Ausdrücke
- Reguläre Grammatiken
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren

5 Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne DFAs
- k1: Ich kenne NFAs
- k1: Ich kenne reguläre Ausdrücke
- k1: Ich kenne reguläre Grammatiken
- k2: Ich kann die Zusammenhänge und Gesetzmäßigkeiten bzgl. der oben genannten Konstrukte an einem Beispiel erklären
- k3: Ich kann für eine Fragestellung DFAs, NFAs, reguläre Ausdrücke, reguläre Grammatiken entwickeln
- k3: Ich kann herausfinden, ob eine Sprache regulär ist
- k3: Ich kann einen DFA entwickeln, der alle Schlüsselwörter, Namen und weitere Symbole einer Programmiersprache akzeptiert

---

Quellen

Aho, A. V., M. S. Lam, R. Sethi, J. D. Ullman, und S. Bansal. 2023. *Compilers: Principles, Techniques, and Tools, Updated 2nd Edition by Pearson*. Pearson India. https://learning.oreilly.com/library/view/compilers-principles-techniques/9789357054881/.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/.

Torczon, L., und K. Cooper. 2012. *Engineering a Compiler*. Morgan Kaufmann. https://learning.oreilly.com/library/view/engineering-a-compiler/9780080916613/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 906479b (lecture: rework outcomes (01_Regular), 2025-08-19)